Strategic Observability

How to Design the Right Stack

By: Moustafa Aboelnaga

SREday | November 7, 2025

About Me



- 11+ years in field
- Engineering Manager in SolarWinds
- Worked in KSA, Dubai, EGYPT and Czech republic.
- Top Mentor

The Challenge — Observability Without Direction

Every organization wants observability, but without a clear strategy, you end up with:



Tool Sprawl

Accumulation of disparate monitoring tools lacking cohesive integration, leading to fragmented views



Data Silos

Observability data residing in isolated systems, preventing holistic understanding of system behavior



Alert Fatigue

Overwhelming volume of alerts, many unactionable or redundant, causing teams to become desensitized



The Root Problem

Organizations often focus on tool acquisition before defining what needs to be observed and why, leading to spiraling costs without clear ROI

Why Strategy Matters

Observability is a **systemic capability** , not just monitoring or logging. Without strategy, it becomes a **cost center** rather than a **value driver** .

Strategic Observability

- Aligns with business objectives
- Focuses on high-value data
- Drives continuous improvement

VS

Non-Strategic Observability

- Tool sprawl and data chaos
- Costs without clear ROI
- Reactive rather than proactive

The Strategic Approach

A strategic approach to observability ensures that every investment directly contributes to organizational goals and enhances overall reliability. It provides a clear framework for prioritizing what to measure and what to ignore.

Introducing the C.L.A.R.I.T.Y. Framework

A comprehensive framework to evaluate, design, and evolve observability at scale



Context
Meaningful data collection

Levels
Appropriate abstraction

Architecture
Service design

Reliability
Metrics for SLOs

Integration
Correlated data

Transparency
Visible reliability

Yield
Operational insights

Each pillar builds upon the previous one, creating a comprehensive approach to observability that supports your organization's reliability goals

C — Context

Define what truly matters to align observability goals with business reliability



Critical Services

Identify core services that directly impact business operations and user experience. Prioritize observability efforts on these key components.

User Journeys

Map the paths users take through your system. Understand the end-to-end experience to identify critical touchpoints for monitoring.

Dependencies

Understand service dependencies to prioritize what to observe and how issues cascade through the system.

Key Takeaway: Context helps focus observability efforts where they matter most, aligning with business reliability goals.

L — Levels

Not every team has the same observability maturity. Define where you are and where you want to go on the maturity ladder.

1

1: Reactive

Responding to incidents after they impact users

2

2: Proactive

Monitoring key performance indicators; some alerting

3

3: Predictive

Using historical data to forecast issues; SLO-driven

4

4: Strategic

Observability as a shared responsibility; continuous improvement

Why It Matters

- Understand your current state
- Define realistic next steps
- Align investments with capacity

↑ Your Journey

Moving through these stages requires both technological adoption and cultural shifts. Each stage builds upon the previous one, creating a solid foundation for advancement.

A — Architecture

Designing your observability architecture is critical for long-term success. The right architecture balances scale, budget, and operational needs.

Choose Architecture

- Match architecture to your scale and budget
- Consider data volume and query patterns
- Plan for future growth and changes

Standardize Telemetry

- Adopt common data formats (OpenTelemetry)
- Implement structured logging
- Use consistent naming conventions

Cost Control

- Implement sampling strategies
- Set retention policies
- Monitor for data quality issues

Key insight: A well-designed architecture is the foundation for sustainable observability that scales with your organization.

R — Reliability Metrics

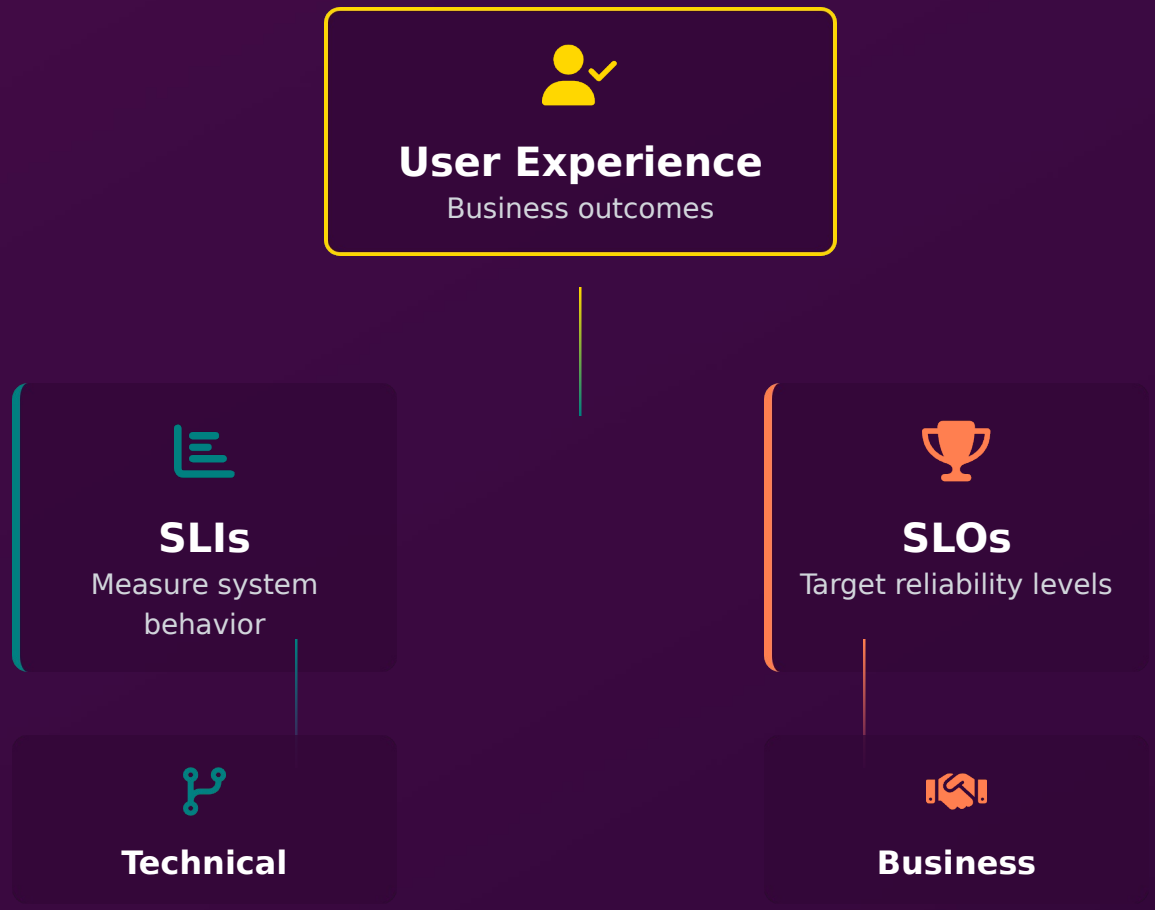
Define SLIs and SLOs that tie to user experience, creating a common language between tech and business for measuring reliability.

SLIs & SLOs

- **SLIs** (Service Level Indicators) are measurements of your system's behavior
- **SLOs** (Service Level Objectives) are targets for your SLIs
- Connect technical metrics to business outcomes

Benefits

- Focus data collection on what matters
- Guide alerting and incident response
- Enable blameless postmortems
- Drive continuous improvement



I — Integration

Connect signals and teams by integrating data into collaboration tools and providing contextual alerts.



Metrics
System health & performance



Logs
Detailed discrete events



Observability Platform
Single pane of glass for all signals



Traces
Request journey through system



Collaboration
Team communication & response



Standardized Formats

Adopt common data formats across services for simplified ingestion



Correlation IDs

Implement across services to link logs, metrics, and traces to specific transactions



Context Propagation

Navigate seamlessly between related data points to understand system behavior

T — Transparency

Make observability everyone's responsibility through transparency

Shared Dashboards

Provide visibility across the organization with accessible dashboards

Accessible Data

Make observability data accessible to those who need it

Blameless Postmortems

Conduct blameless postmortems to learn from incidents

Building Trust Through Transparency

- Improves collaboration
- Builds trust
- Enhances learning

"Transparency is the foundation of trust in observability."

Y — Yield

Measure the impact of observability to prove business value and justify investments.

MTTR Improvement

Quantify how much observability reduces mean time to resolution for incidents

Direct impact on business continuity

Alert Noise Reduction

Measure reduction in alert volume and improvement in signal-to-noise ratio

Less fatigue, more actionable alerts

Cost Efficiency

Track infrastructure cost savings through improved resource utilization

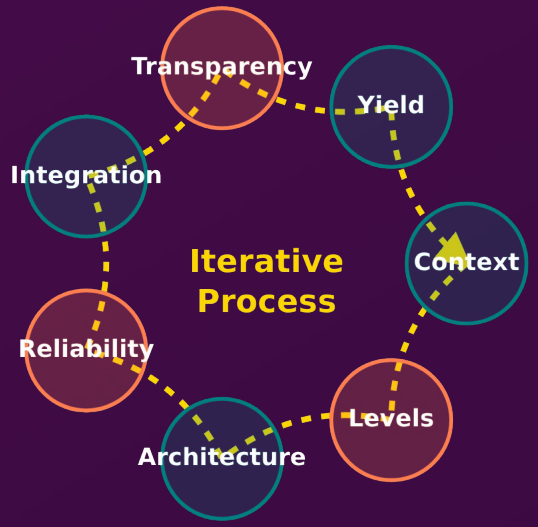
Proactive identification of inefficiencies

Measuring Yield

Establish clear metrics before building your observability stack to ensure you're collecting meaningful data that demonstrates business impact

Bringing It All Together

The C.L.A.R.I.T.Y. Framework is **iterative** , not a checklist. It's a continuous cycle of improvement.



Start Small

Begin with a focused approach on critical systems. Don't try to observe everything at once.

Measure Value

Define success metrics for your observability practice. Focus on ROI and business impact.

Evolve Practice

Continuously refine your approach based on what you learn. Grow your observability capabilities over time.

> *Observability is not a destination, but a continuous journey of improvement.*

How to Adopt C.L.A.R.I.T.Y. (Without Causing Chaos) 😄

🪜 Step-by-Step Adoption Path

1 Start with Awareness 💡

- Begin with the why, not the what
- Host a short session: "What does clarity mean for us?"
- Share relatable pain points

2 Pilot First, Don't Preach 🧪

- Choose one service as a pilot
- Apply C.L.A.R.I.T.Y. pillars
- Show early wins (20% faster MTTR)
- Success stories sell better than slides

3 Iterate and Celebrate ⚙🎉

- Review what worked, improve what didn't
- Make small, visible wins public
- Encourage Observability Champions

4 Scale the Culture 🚀

- Bake observability into sprints & design reviews
- Reward proactive detection
- Make observability a mindset, not a task



Step 0:
Convince management it's not just another logging tool. 😄

💬 *"Start small, learn fast, share loudly — that's how you bring clarity without chaos."*

Challenges You'll Face (and How to Tame Them)

The "We Already Have Dashboards" Syndrome

Challenge: Everyone thinks observability = charts.

Fix: Show that observability is about understanding, not visualizing. Link it to reliability metrics and SLOs — dashboards come after direction.

Alert Fatigue & Noise

Challenge: Engineers tune out alerts.

Fix: Use the Simplify and Reliability Metrics pillars. Alert on user impact, not on every CPU spike.

Budget Reality Check

Challenge: Fancy tools ≠ guaranteed insight.

Fix: Use open-source where possible; measure yield before scaling spend. Prove value first — "Less cost, more clarity."

Cultural Resistance

Challenge: "That's the SRE team's job."

Fix: Adopt the Transparency and Integration pillars. Show that shared visibility = shared success. Celebrate teams that fix visibility gaps before incidents.

The Perfection Trap

Challenge: Trying to monitor everything, all at once.

Fix: Focus on what matters. Start with one service, one SLO, one clear win — evolve from there.



"You don't need perfect observability — you need useful observability."

"Because no great framework survives first contact with production."

C.L.A.R.I.T.Y. Scoreboard: Measuring Success



Framework Success Metrics

Pillar	What Success Looks Like	Example Metrics	Status / Target
C Context	Teams know what matters to observe	% of services with SLIs/SLOs	↑ 80–90%
L Levels	Teams improve observability maturity	Teams progressing 1 level/quarter	Improving
A Architecture	Unified observability stack	% of standardized telemetry	↑ 75%
R Reliability	Connection between SLOs and incidents	% of incidents tied to SLO breaches	↑ 90%
I Integration	Metrics, logs, and traces correlate	% of alerts with trace correlation	↑ 80%
T Transparency	Shared visibility and participation	% of teams using shared dashboards	↑ 70–85%
Y Yield	Measurable value and cultural change	↓ MTTR, ↑ satisfaction, ↓ noise	Positive ROI

Overall Framework Success Indicators

Adoption
Teams using framework
100% by Year 1

Engagement
Monthly reviews
≥1 per team

Maturity Growth
Org maturity level
+1 per quarter

Cultural Shift
Survey score
4.5 / 5

Sustainability
Cost-to-value ratio
↓ 20–30%

> *"The C.L.A.R.I.T.Y. Framework is not about tools — it's about behavior and improvement."*
> When this scoreboard turns green, your observability isn't just running — it's working.

Key Takeaways

"*Observability must start with strategy, not tools.*"

The C.L.A.R.I.T.Y. Framework

C — Context
Instrument with meaningful context

L — Levels
Measure from request to resource

A — Architecture
Design for observability

R — Reliability
Align with SLOs

I — Integration
Connect signals across systems

T — Transparency
Share insights across teams

Y — Yield
Measure ROI of observability investments

Strategy Before Tools

Define your reliability goals and SLOs before selecting tools. This prevents tool sprawl and ensures your observability stack serves your organization's vision.

Build for the Long Haul

Foster a learning culture with blameless postmortems and data-driven decisions. This creates a shared responsibility for reliability.

"*Observability is not just about the volume of data collected, but the strategic collection and intelligent analysis of the right data.*"

Thank You / Discussion



Thank you for your attention!

Q & A